Application Note

FESTO

VTUX.dll C# Library
PC Based Control Via Modbus TCP/IP

This Application Note explains the structure of VTUXlib.dll library including properties and methods and use this to control VTUX via Modbus TCP/IP. The library supports to read/write most data behind CPX-AP-.-EP node.

CPX-AP-A-EP-M12

CPX-AP-I-EP-M12

20260520

Title ..... Title
Version 1.10
Document no. 20261012
Original ..... en
Author ..... Festo

Last saved 14.10.2026

General information:

This document is intended for qualified, trained and instructed professionals. The data provided in this document are no guaranteed specifications, in particular with regard to functionality, condition or quality in the legal sense. The information in this document is intended only as simple indications for the implementation of a specific, hypothetical application, and in no way as a substitute for the operating instructions of the respective manufacturers or the design and testing of the respective application by the user. The respective operating instructions for Festo products can be found under www.festo.com. The user of this document must ensure that each function described herein works properly in his application. The user remains solely responsible for his or her own use of this document, even by studying this document and using the information mentioned therein. This also applies to any software (in source code and/or object code) that is made available to the user as an appendix to this document.

Rights of use of the software:

If software is made available to the user as an appendix to this document, the user is granted a simple and unlimited right of use hereto. This right also includes the processing and distribution of the software to third parties in edited or unedited form. The User is not permitted to use the name "Festo" to endorse or promote products derived from the Software without express prior written permission.

Software is provided to the user "as is". Festo does not assume any warranty or guarantee with regard to software. Festo's liability for damages of any kind arising from the use of the software is limited to intent and gross negligence.

Legal Notices:

©Festo SE & Co. KG, all rights reserved. A change in content and form is only permitted with the express written consent of Festo SE & Co. KG. Festo grants the user the right to reproduce this document in a form that remains unchanged in terms of content and form and to pass it on to third parties.

© (Festo SE & CO. KG, D-73726 Esslingen, 2026)
Internet: http://www.festo.com

Table of contents

1. Objective, Features And Idea Behind 4

1.1. Objective 4

1.2. Features 4

1.3. Idea behind 4

2. Components/Software used 5

2.1. Hardware overview 5

2.1.1. Completed testing unit 5
2.1.2. CPX-AP-A-EP-M12 5
2.1.3. VABX-A-S-BV-V4A 6

2.2. Topology 6

3. Software set up 7

3.1. Basic methods and properties 7

3.2. Add VTUXlib.dll 7

3.3. Example with Console Application 7

3.3.1. Set 6 coils of VTUX. 7
3.3.2. Example: Read module information 10

3.3.3. Example With WinForms Application 11

4. Library 14

4.1. Initialization 14

4.2. Read Digital Input 14

4.3. Read Analog Input 14

4.4. Read Output Coils 15

4.5. Write Output 15

4.6. Write system parameter 17

4.7. Read parameter mailbox. 17

4.8. Read general diagnosis information 19

4.9. Read module diagnosis information 19

4.10. Read general information 20

4.11. Read system parameter. 21

4.12. Read module information. 22

4.13. Read IO-Link Mailbox 25

4.14. Read block data. 27

4.15. Write IO Link Mailbox 29

4.16. Write Parameter Mailbox 30

1. Objective, Features And Idea Behind

1.1. Objective

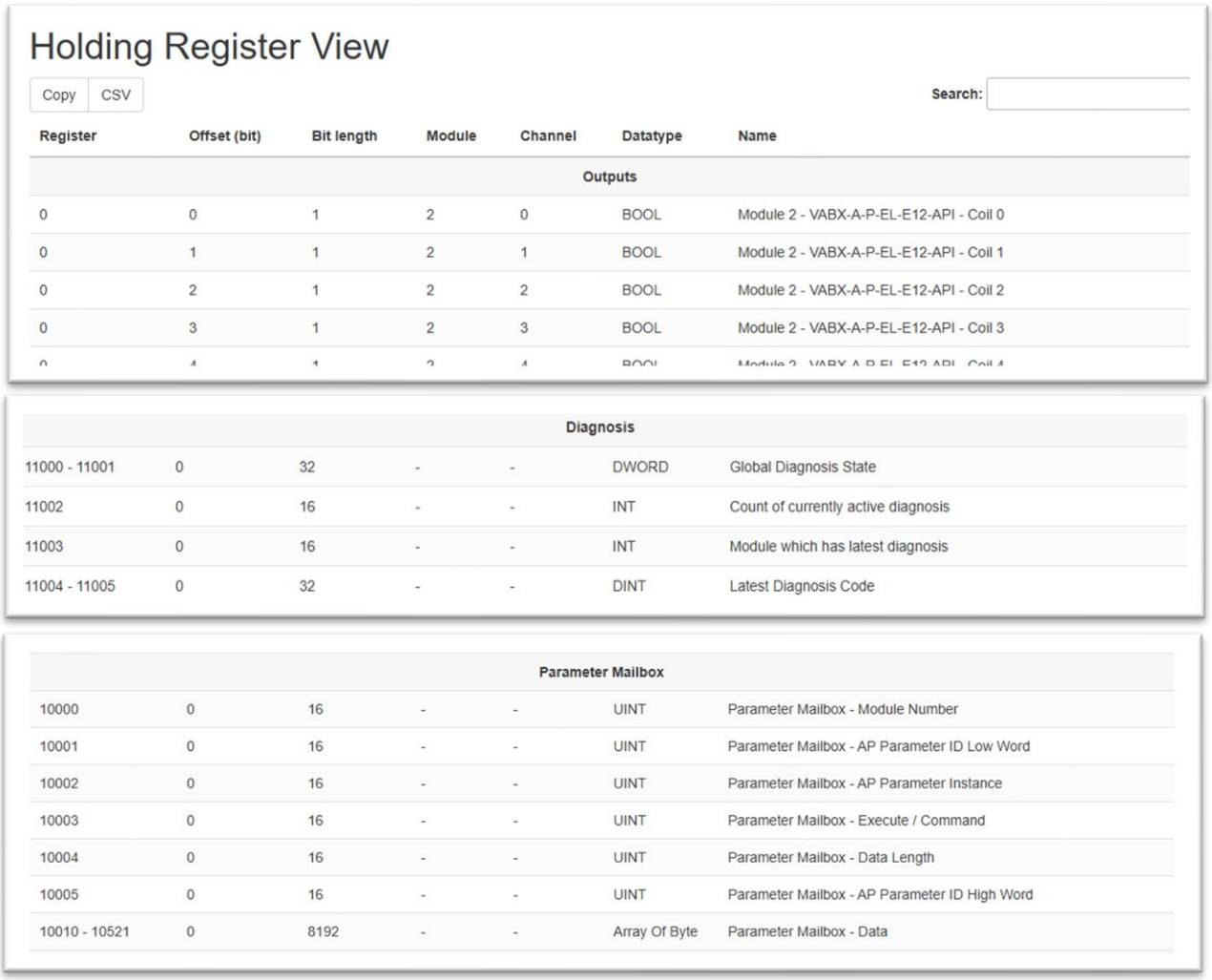
Provide a library that supports programmers to communicate with VTUX valve terminals quickly and reliably

1.2. Features

- Full Modbus TCP/IP support over ethernet with C#.
- Access to all available registers (inputs, outputs, diagnosis, module information, etc..).
- Category reading data as blocks.
- Set up system parameters (Timeout, SNMP/webserver enable, etc..).

1.3. Idea behind

- Access to all registers based on CPX-AP-A/AP-I-EP provided.
- Category reading/writing data.



Holding Register View

Copy CSV Search:

Register	Offset (bit)	Bit length	Module	Channel	Datatype	Name
Outputs						
0	0	1	2	0	BOOL	Module 2 - VABX-A-P-EL-E12-API - Coil 0
0	1	1	2	1	BOOL	Module 2 - VABX-A-P-EL-E12-API - Coil 1
0	2	1	2	2	BOOL	Module 2 - VABX-A-P-EL-E12-API - Coil 2
0	3	1	2	3	BOOL	Module 2 - VABX-A-P-EL-E12-API - Coil 3
Diagnosis						
11000 - 11001	0	32	-	-	DWORD	Global Diagnosis State
11002	0	16	-	-	INT	Count of currently active diagnosis
11003	0	16	-	-	INT	Module which has latest diagnosis
11004 - 11005	0	32	-	-	DINT	Latest Diagnosis Code
Parameter Mailbox						
10000	0	16	-	-	UINT	Parameter Mailbox - Module Number
10001	0	16	-	-	UINT	Parameter Mailbox - AP Parameter ID Low Word
10002	0	16	-	-	UINT	Parameter Mailbox - AP Parameter Instance
10003	0	16	-	-	UINT	Parameter Mailbox - Execute / Command
10004	0	16	-	-	UINT	Parameter Mailbox - Data Length
10005	0	16	-	-	UINT	Parameter Mailbox - AP Parameter ID High Word
10010 - 10521	0	8192	-	-	Array Of Byte	Parameter Mailbox - Data

Figure 1.1 VTUX holding registors

2. Components/Software used

Type/Name	Version Software/Firmware	Date of manufacture
Microsoft Visual Studio Professional 2022 (64 bit)	17.14.13	
.Net Framework	4.7.2	
CPX-AP-A-EP-M12	1.7.4	
VABX-A-S-BV-V4A	1.122.7	

Table 2.1: Components/Software used

2.1. Hardware overview

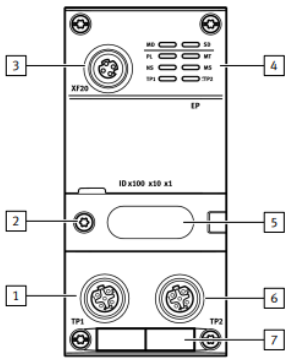
2.1.1. Completed testing unit



Figure 2.2: VTUX-A-S-APA-USAJKCG-XLRV01J4AXR/70E-ECQA-X

2.1.2. CPX-AP-A-EP-M12

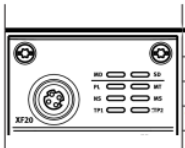
Product design



1 EtherNet/IP network connection [TP1]
2 Rotary switch screw cover
3 AP system communication connection [XF20]
4 LED indicator
5 Rotary switch cover
6 EtherNet/IP network connection [TP2]
7 Inscription label

Fig. 1: Product design

LED indicators



LED	Meaning
MD	Module diagnostics [MD] (green/red)
PL	Load supply [PL] (green/red)
NS	Network status [NS] (green/red)
TP1	Network connection [TP1] (green)
SD	System diagnostics [SD] (green/red)
MT	Maintenance [MT] (yellow)
MS	Module status [MS] (green/red)
TP2	Network connection [TP2] (green)

Tab. 5: LED indicators

Figure 2.3: CPX-AP-A-EP-M12 interface

2.1.3. VABX-A-S-BV-V4A



Figure 2.3: VABX-A-S-V

2.2. Topology



Figure 2.4: Topology uses for the test

3. Software set up

3.1. Basic methods and properties

Namespace	VTUXlib
Class	VTUX

Table 3.1 Namespace and class.

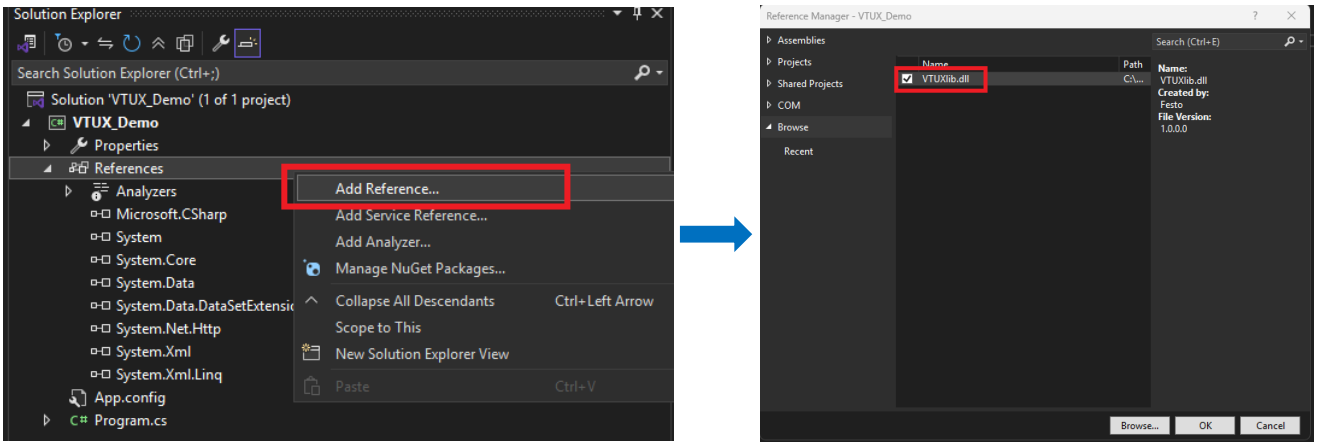
No.	Property Name	Description
1	ModuleCount	Count the number of module on the network
2	ModuleCode	Point out module code
3	InputSize	Input size of module
4	OutputSize	Output size of module
5	InputChannel	Number of module input channel
6	OutputChannel	Number of module output channel
7	StartInputAddress	Module start input address
8	StartOutputAddress	Module start output address
9	IsConnected	Get the connection status

Table 3.2 Properties

For details of method using in this library please refer to section “4.Library”

3.2. Add VTUXlib.dll

Create a new project with C# and add VTUXlib.dll.



3.3. Example with Console Application

3.3.1. Set 6 coils of VTUX.

Call VTUXlib namespace in project

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using VTUXlib;
namespace VTUX_Demo
{
```

Declare an object and connect to VTUX

```
VTUX Com = new VTUX("192.168.1.100", 502);
```

Call *Connect()* method to establish the connection to device. After connection is done, we need to call *initialization()*. This method will collect and set up all device information such as: Input channel, output channel, input size, output size, start input address, start output address, count module available and module code. This must be called once before accessing to other methods.

```
Com.Connect();
if (Com.IsConnected)
{
    Console.WriteLine("Connected to VTUX");
    Com.Initialization();
}
else
{
    Console.WriteLine("Failed to connect to VTUX");
}
```

Call Set_Multi_Output_Coils () to set 6 coils from module number 2 in VTUX

```
while (Com.IsConnected)
{
    // Set 6 coils of the output module 2
    Com.Set_Multi_Output_Coils(2, 0, 6);
    Thread.Sleep(50);
}
```

Completed code

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using VTUXlib;
namespace VTUX_Demo
{
  internal class Program
  {
    static void Main(string[] args)
    {
      VTUX Com = new VTUX("192.168.1.100", 502,1000);
      Com.Connect();
      if (Com.IsConnected)
      {
        Console.WriteLine("Connected to VTUX");
        Com.Initialization();
      }
```

```
            else
            {
              Console.WriteLine("Failed to connect to VTUX");
            }
            while (Com.IsConnected)
            {
              // Set 6 coils of the output module 2
              Com.Set_Multi_Output_Coils(2, 0, 6);
              Thread.Sleep(50);
            }
          }
        }
      }
```

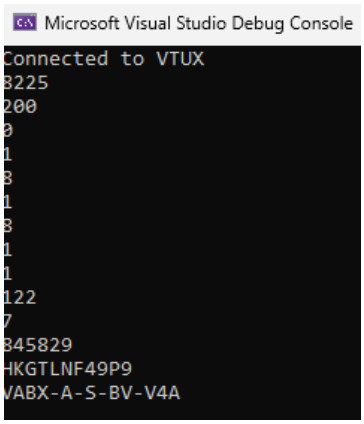
Read actual output channel and set all of coils to ON.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using VTUXlib;
namespace VTUX_Demo
{
  internal class Program
  {
    static void Main(string[] args)
    {
      VTUX Com = new VTUX("192.168.1.100", 502,1000);
      Com.Connect();
      if (Com.IsConnected)
      {
        Console.WriteLine("Connected to VTUX");
        Com.Initialization();
      }
      else
      {
        Console.WriteLine("Failed to connect to VTUX");
      }
      while (Com.IsConnected)
      {
        ushort outputChannel = Com.OutputChannel[2];
        Com.Set_Multi_Output_Coils(2, 0, outputChannel);
        Thread.Sleep(50);
      }
    }
  }
}
```

3.3.2. Example: Read module information

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using VTUXlib;
namespace VTUX_Demo
{
  internal class Program
  {
    static void Main(string[] args)
    {
      VTUX Com = new VTUX("192.168.1.100", 502,1000);
      Com.Connect();
      if (Com.IsConnected)
      {
        Console.WriteLine("Connected to VTUX");
        Com.Initialization();
      }
      else
      {
        Console.WriteLine("Failed to connect to VTUX");
      }
      while (Com.IsConnected)
      {
        List<object> Ob =  Com.Read_Block_Module_Information(2);
        foreach (object item in Ob)
        {
          Console.WriteLine(item);
        }
        break;
      }
    }
  }
}
```

Result:



3.3.3. Example With WinForms Application

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using VTUXlib;
namespace VTUXWinformDemo
{
  public partial class Form1 : Form
  {

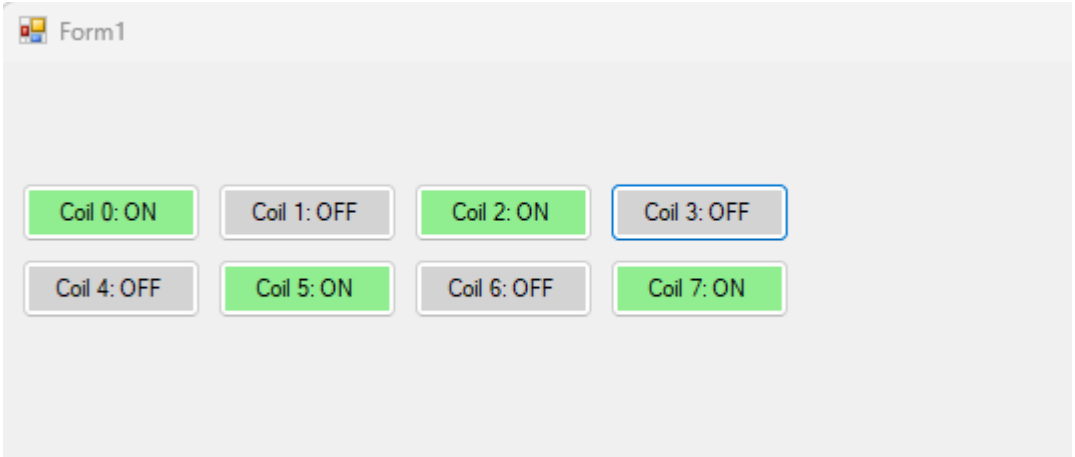
    VTUX Com = new VTUX("192.168.1.99", 502,1000);
        public int idx; // made public so other classes/forms can access selected coil index
    private bool newState = false; // example state to write
    private Button[] dynamicButtons;
    private bool[] dynamicButtonStates;
    private int DynamicButtonCount;
    public Form1()
    {
      InitializeComponent();
      Com.Connect();
      if (Com.IsConnected)
      {
        Com.Initialization();
        DynamicButtonCount = Com.OutputChannel[2];
      }
      else
      {
        // do nothing
      }
      InitializeDynamicButtons();
    }
```

```
private void InitializeDynamicButtons()
{
  // layout settings
  int cols = 4;
  int btnWidth = 90;
  int btnHeight = 30;
  int margin = 8;
  int startX = 10;
            int startY = 60; // place below top controls; adjust to fit your form
  dynamicButtons = new Button[DynamicButtonCount];
  dynamicButtonStates = new bool[DynamicButtonCount];
  for (int i = 0; i < DynamicButtonCount; i++)
  {
    var btn = new Button();
    btn.Width = btnWidth;
    btn.Height = btnHeight;
    int col = i % cols;
    int row = i / cols;
    btn.Left = startX + col * (btnWidth + margin);
    btn.Top = startY + row * (btnHeight + margin);
    btn.Text = $"Coil {i}: OFF";
    btn.Name = $"btnDyn{i}";
    btn.Tag = i; // store index
    btn.BackColor = Color.LightGray;
    btn.Click += DynamicButton_Click;
    dynamicButtons[i] = btn;
    this.Controls.Add(btn);
  }
}
private void DynamicButton_Click(object sender, EventArgs e)
{
  if (!(sender is Button btn)) return;
  if (!(btn.Tag is int tagIndex)) return;
            // assign tagIndex to the public field so timer1_Tick and other code see the selected
            index
  idx = tagIndex;
  // toggle state
  dynamicButtonStates[idx] = !dynamicButtonStates[idx];
  newState = dynamicButtonStates[idx];
  // update UI
  btn.Text = $"Coil {idx}: {(newState ? "ON" : "OFF")}";
  btn.BackColor = newState ? Color.LightGreen : Color.LightGray;
}
private void Form1_Load(object sender, EventArgs e)
{
}

private void timer1_Tick_1(object sender, EventArgs e)
{
```

```
            // write to VTUX if connected
            try
            {
              if (Com != null && Com.IsConnected)
              {
                // Example: Write_Output_Coil(slot, coilIndex, state)
                            // Adjust parameters to match your VTUXlib API and addressing
                Com.Write_Single_Output_Coil(2, (ushort)idx, newState);
              }
              else
              {
                // optionally show disconnected state
                            MessageBox.Show("VTUX not connected", "Error", MessageBoxButtons.OK,
                MessageBoxIcon.Warning);
              }
            }
            catch (Exception ex)
            {
                        MessageBox.Show($"Write failed: {ex.Message}", "Error", MessageBoxButtons.OK,
              MessageBoxIcon.Error);
            }
          }
        }
      }
```

Result:



4. Library

4.1. Initialization

No.	Method Name	Description
1	Initialization()	- This method should be called after the connection is established and before any read/write operation. - It initializes the module information and address mapping based on the data read from the device. The method first reads the module count, then allocates arrays for module information and address mapping. - It iterates through each module to read its details such as module code, input/output size, and channel count. Based on the input/output size of each module, it calculates the starting addresses for input and output registers for each module. - This initialization is crucial for correctly addressing the registers when performing read/write operations for different modules.

4.2. Read Digital Input

No.	Method Name	Description
1	Read_Single_Digital_Input (ushort moduleNo, ushort inputNo)	- Read a digital input that is mapped inside holding registers (same packing as outputs). - ModuleNo is 1-based; inputNo is zero-based index within the module.
2	Read_Multi_Digital_Inputs (ushort moduleNo, ushort startInputNo, ushort length)	- Read multiple digital inputs that are packed inside holding registers (same packing as outputs). - ModuleNo is 1-based; startInputNo is zero-based index within the module; length is number of inputs to read. - Returns boolean array of length 'length' with states for each requested input.
3	Read_Block_Digital_Input_Module(ushort moduleNo)	- This method returns the parameters related to the digital input module in a block for the specified module. - User can use this method to read multiple parameters related to the digital input module at once which can be more efficient than reading each parameter separately.

4.3. Read Analog Input

No.	Method Name	Description
1	Read_Single_Analog_Input (ushort moduleNo, ushort channelNo)	- Read a single analog input channel value for the specified module.

		- ModuleNo is 1-based; channelNo is zero-based index within the module. - Returns a signed 16-bit value..
2	Read_Single_Scale_Analog_Input (ushort moduleNo, ushort channelNo, ushort lowRaw, ushort highRaw, float lowScale, float highScale)	-Read a single analog input channel value for the specified module and apply linear scaling based on provided raw and scale parameters.
3	Read_Block_Analog_Input_Module (ushort moduleNo)	- Read block module - analog input. This method returns the parameters related to the analog input module in a block for the specified module.

4.4. Read Output Coils

No.	Method Name	Description
1	Read_Single_Output_Coil (ushort moduleNo, ushort coilNo)	- This method reads the state of a single output coil by computing the target register and bit position based on the module number and coil number, - Reading the current value of the holding register that contains the target coil, and returning the boolean state of the specific bit for the target coil.
2	Read_Multi_Output_Coils (ushort moduleNo, ushort startCoilNo, ushort length)	- This method reads the state of multiple output coils that are packed inside holding registers. - It computes the target register range that covers the specified coil range, reads the current values of those holding registers, and extracts the boolean states for each requested coil based on their respective bit positions within the registers.

4.5. Write Output

No.	Method Name	Description
1	Write_Single_Output_Coil (ushort moduleNo, ushort coilNo, bool value)	- This method sets the specified coil to 1. - It reads the current register value that contains the target coil, sets only the bit for the target coil to 1 while preserving other bits, and writes back the modified register value. - This ensures that only the specified coil is set without affecting other coils in the same register.
2	Write_Multi_Output_Coils (ushort moduleNo, ushort startCoilNo, bool[] values)	- This methods sets multiple coils starting from the specified coil number based on the values in the input array. - The method reads the current register values that contain the target coils, sets or resets only the bits for the target coils based on the input values while preserving other bits, and writes back the modified register values.

		- This ensures that only the specified coils are set or reset according to the input values without affecting other coils in the same registers.
3	Set_Single_Output_Coil (ushort moduleNo, ushort coilNo)	- This method resets the specified coil by setting the corresponding bit to 0 in the holding register. - It reads the current register value, clears only the target bit while preserving other bits, and writes back the modified register value. - This ensures that only the specified coil is reset without affecting other coils in the same register.
4	Set_Multi_Output_Coils (ushort moduleNo, ushort startCoilNo, ushort length)	- This methods sets multiple coils starting from the specified coil number for the specified length. - The method reads the current register values that contain the target coils, sets only the bits for the target coils to 1 while preserving other bits, and writes back the modified register values. - This ensures that only the specified coils are set without affecting other coils in the same registers.
5	Reset_Single_Output_Coil (ushort moduleNo, ushort coilNo)	- This method resets the specified coil by setting the corresponding bit to 0 in the holding register. - It reads the current register value, clears only the target bit while preserving other bits, and writes back the modified register value. - This ensures that only the specified coil is reset without affecting other coils in the same register.
6	Reset_Multi_Output_Coils (ushort moduleNo, ushort startCoilNo, ushort length)	- This method resets multiple coils starting from the specified coil number for the specified length. - The method reads the current register values that contain the target coils, clears only the bits for the target coils while preserving other bits, and writes back the modified register values. - This ensures that only the specified coils are reset without affecting other coils in the same registers.
7	Toggle_Single_Output_Coil (ushort moduleNo, ushort coilNo)	- This method toggles the output coil by reading the current value of the register that contains the target coil, toggling only the specific bit for the target coil while preserving other bits, and writing back the modified register value. - This ensures that only the specified coil is toggled without affecting other coils in the same register. - The method computes the target register and bit position based on the module number and coil number, and performs a read-modify-write operation to toggle the coil state.

8	Toggle_Multi_Output_Coils (ushort moduleNo, ushort startCoilNo, ushort length)	- This method toggles multiple coils starting from the specified coil number for the specified length. - The method reads the current register values that contain the target coils, toggles only the bits for the target coils while preserving other bits, and writes back the modified register values. - This ensures that only the specified coils are toggled without affecting other coils in the same registers.

4.6. Write system parameter

No.	Method Name	Description
1	Write_Modbus_Timeout(short value)	- This method writes the Modbus timeout value into the corresponding register. - The timeout value is represented in milliseconds and it defines the time duration that the device will wait for a response after sending a Modbus request before considering it as a timeout. - Setting an appropriate timeout value is important to ensure reliable communication, especially in environments with variable network conditions or when communicating with devices that may have longer response times.
2	Write_Webserver_Enable(bool value)	- This method writes the enable/disable value for the webserver function into the corresponding register.
3	Write_Webserver_Access_Enable (bool value)	- This method writes the enable/disable value for the SNMP function into the corresponding register.
4	Write_SNMP_Enable(bool value)	- This method writes the enable/disable value for the SNMP function into the corresponding register.

4.7. Read parameter mailbox.

No.	Method Name	Description
1	Read_Parameter_Mailbox_ModuleNo()	- This method returns the parameter value for module parameter. - The specific definition of the module parameter and how to use it can be found in the device manual. - User can write the corresponding module parameter into this register and execute the command to read/write the parameter value based on the parameter information and data in the mailbox.
2	Read_Parameter_Mailbox_AP_Para_Low_Word()	- This method returns the low word for application parameter.

		- The complete application parameter is represented by combining both low word and high word. The way to combine the low word and high word to get the complete parameter value
3	Read_Parameter_Mailbox_AP_Para_Instance()	- This method returns the instance number for the application parameter. - The instance number is used when there are multiple instances for the same type of application parameter. - The way to use the instance number depends on the specific parameter and the definition can be found in the device manual.
4	Read_Parameter_Mailbox_Execute_Command()	- This method returns the command for parameter read/write operation. - The command definition can be found in the device manual. User can write the corresponding command into this register to execute the parameter read/write operation based on the parameter information and data in the mailbox. - After the command is executed, the device will clear the command register to 0 and user can check this register to know if the command execution is completed.
5	Read_Parameter_Mailbox_Data_Length()	- This method returns the data length for the parameter read/write command. The data length is represented in byte unit.
6	Read_Parameter_Mailbox_AP_Para_High_Word()	- This methods returns the high word for application parameter. - The complete application parameter is represented by combining both low word and high word. - The way to combine the low word and high word to get the complete parameter value depends on the specific parameter and the definition can be found in the device manual.
7	Read_Parameter_Mailbox_Data()	- This method reads the data in parameter mailbox which is used for both module parameter and application parameter. - The data format and content are specific to the parameter and the definition can be found in the device manual.
8	Read_AP_Parameter (ushort ModuleNo, ushort AP_Para_Low_Word, ushort AP_Para_High_Word, ushort AP_Para_Instance, ushort Data_Length)	- This method reads a specified number of bytes from the application parameter mailbox for a given module, parameter address, and instance. - It handles command execution and waits for completion before retrieving the data.

4.8. Read general diagnosis information

No.	Method Name	Description
1	Read_General_Diagnosis_Global_Diagnosis_State()	- Read general diagnosis - global diagnosis state. - This method returns the global diagnosis state which is represented in bitmask format. - The bit definition for the diagnosis state can be found in the device manual. - User can use this global diagnosis state for quick check if there is any active diagnosis and the severity level of the active diagnosis without checking each individual diagnosis.
2	Read_General_Diagnosis_Count_Current_Active_Diagnosis()	- Read general diagnosis - count current active diagnoisis. - This method returns the count of current active diagnosis. - The definition of diagnosis and the corresponding diagnosis number can be found in the device manual.
3	Read_General_Diagnosis_Lastest_Diagnosis()	- Read general diagnosis - latest diagnosis. This methods returns the diagnosis number for the latest diagnosis that just got activated. - If there is no active diagnosis, the method returns 0. The diagnosis number is specific to the device and the definition can be found in the device manual.
4	Read_General_Diagnosis_Lastest_Code()	- This methods returns the diagnosis code for the latest diagnosis that just got activated. - If there is no active diagnosis, the method returns 0. The diagnosis code is specific to the device and the definition can be found in the device manual.

4.9. Read module diagnosis information

No.	Method Name	Description

1	Read_Module_Diagnosis_Module(ushort moduleNo)	- Read module diagnosis - module. This method returns the module number for the active diagnosis with highest severity level. - If there is no active diagnosis, the method returns 0. The module number is specific to the device and the definition can be found in the device manual.
2	Read_Module_Diagnosis_Submodule(ushort moduleNo)	- Read module diagnosis - submodule. This method returns the submodule number for the active diagnosis with highest severity level. - If there is no active diagnosis, the method returns 0. The submodule number is specific to the module and the definition can be found in the device manual.
3	Read_Module_Diagnosis_Channel(ushort moduleNo)	- Read module diagnosis - channel. This methods returns the channel number related to the active diagnosis with highest severity level. - If there is no active diagnosis or the diagnosis is not related to any specific channel, the method returns 0.
4	Read_Module_Diagnosis_Present_State(ushort moduleNo)	- Read module diagnosis - present state. This methods returns the present state for the active diagnosis with highest severity level. - If there is no active diagnosis, the method returns 0. The present state is represented in bitmask format and the bit definition is diagnosis code specific which can be found in the device manual.
5	Read_Module_Diagnosis_Module_Diagnosis_State(ushort moduleNo)	- Read module diagnosis - diagnosis state. This method returns the diagnosis state for the active diagnosis with highest severity level. - If there is no active diagnosis, the method returns 0. The diagnosis state is represented in bitmask format and the bit definition is diagnosis code specific which can be found in the device manual.
6	Read_Module_Diagnosis_Diagnosis_Code(ushort moduleNo)	- Read module diagnosis - diagnosis code. This method returns the diagnosis code for the active diagnosis with highest severity level. - If there is no active diagnosis, the method returns 0.

4.10. Read general information

No.	Method Name	Description
1	Read_General_Info_Module_Count()	- Read general info - module count. This method returns the total number of modules (including main module and extension modules) present in the system.

4.11. Read system parameter.

No.	Method Name	Description
1	Read_System_Parameter_Modbus_Timeout()	- Read system parameter - Modbus timeout. This method returns the Modbus timeout value in millisecond. - The Modbus timeout is the time duration that the device waits for receiving a complete Modbus request or sending a complete Modbus response before it considers the communication as failed. User can adjust the Modbus timeout value based on the network condition to achieve better communication performance. For example, if there is a network delay, user can increase the Modbus timeout value to avoid unnecessary communication failure.
2	Read_System_Parameter_Webserver_Enable()	- Read system parameter - webserver enable. This method returns the enable status for webserver. - If the webserver is enabled, the method returns true; if the webserver is disabled, the method returns false. - The webserver provides a built-in webpage for user to monitor and control the device. User can enable or disable the webserver based on the actual need. For example, user can disable the webserver for better security if there is no need to use the webserver.
3	Read_System_Parameter_Webserver_Enable()	- Read system parameter - webserver write access enable. This method returns the enable status for webserver write access. - If the webserver write access is enabled, the method returns true; if the webserver write access is disabled, the method returns false. - The webserver write access allows user to make changes to the device configuration through the webserver webpage. User can enable or disable the webserver write

		access based on the actual need. For example, user can disable the webserver write access to prevent unauthorized changes to the device configuration through the webserver.
4	Read_System_Parameter_SNMP_Enable()	- read system parameter - SNMP enable. This method returns the enable status for SNMP. - If SNMP is enabled, the method returns true; if SNMP is disabled, the method returns false. - SNMP (Simple Network Management Protocol) is a network protocol used for monitoring and managing network devices. - User can enable or disable SNMP based on the actual need. For example, user can disable SNMP if there is no need to use SNMP for device monitoring and management to reduce potential security risk.
5	Read_System_Parameter_Webserver_Write_Access_Enable()	-Read system parameter - webserver write access enable. This method returns the enable status for webserver write access. - If the webserver write access is enabled, the method returns true; if the webserver write access is disabled, the method returns false. - The webserver write access allows user to make changes to the device configuration through the webserver webpage. User can enable or disable the webserver write access based on the actual need. For example, user can disable the webserver write access to prevent unauthorized changes to the device configuration through the webserver.

4.12. Read module information.

No.	Method Name	Description
1	Read_Module_Info_Module_Code (ushort moduleNo)	- Read module info - module code. This method returns the module code for the specified module.

		- The module code is specific to the device and the definition can be found in the device manual. - User can use the module code to identify the type of the module and get more information about the module from the device manual.
2	Read_Module_Info_Module_Class (ushort moduleNo)	- read module info - module class. This method returns the module class for the specified module. - The module class is specific to the device and the definition can be found in the device manual. - User can use the module class to identify the category of the module and get more information about the module from the device manual.
3	Read_Module_Info_Com_Profile (ushort moduleNo)	- read module info - communication profile. This method returns the communication profile for the specified module. - The communication profile defines the communication capabilities of the module such as supported protocols, data rates, etc. - The specific definition of the communication profile can be found in the device manual. User can use the communication profile to understand the communication capabilities of the module and configure the communication settings accordingly.
4	Read_Module_Info_Input_Size (ushort moduleNo)	- Read module info - input size. This method returns the input size for the specified module. - The input size defines the number of input channels or data points for the module. The specific definition of the input size can be found in the device manual. - User can use the input size to understand how many input channels or data points are available for the module and configure the system accordingly.
5	Read_Module_Info_Input_Channel (ushort moduleNo)	- read module info - input channel. This method returns the starting channel number for the input channels of the module. - The channel number is used to identify each input channel for the module. The specific definition of the channel number can be found in the device manual. - User can use the channel number to understand how to map the input channels of the module in the system and access the input data accordingly.
6	Read_Module_Info_Output_Size (ushort moduleNo)	- read module info - output size. This method returns the output size for the specified module. - The output size defines the number of output channels or data points for the module. - The specific definition of the output size can be found in the device manual.

		- User can use the output size to understand how many output channels or data points are available for the module and configure the system accordingly.
7	Read_Module_Info_Output_Channel (ushort moduleNo)	- Read module info - output channel. This method returns the starting channel number for the output channels of the module. - The channel number is used to identify each output channel for the module. The specific definition of the channel number can be found in the device manual. - User can use the channel number to understand how to map the output channels of the module in the system and access the output data accordingly.
8	Read_Module_Info_Hardware_Vesion (ushort moduleNo)	- Read module info - hardware version. This method returns the hardware version for the specified module. - The hardware version is specific to the device and the definition can be found in the device manual. - User can use the hardware version to understand the hardware revision of the module which can be helpful for troubleshooting and support.
9	Read_Module_Info_Firmware_Version_ Major(ushort moduleNo)	- Read module info - firmware version. This method returns the firmware version for the specified module. - The firmware version is specific to the device and the definition can be found in the device manual. - User can use the firmware version to understand the firmware revision of the module which can be helpful for troubleshooting, support and firmware update.
10	Read_Module_Info_Firmware_Version_ Minor(ushort moduleNo)	- Read module info - firmware version. This method returns the firmware version for the specified module. - The firmware version is specific to the device and the definition can be found in the device manual. - User can use the firmware version to understand the firmware revision of the module which can be helpful for troubleshooting, support and firmware update.
11	Read_Module_Info_Firmware_Version_ Patch(ushort moduleNo)	- Read module info - firmware version. This method returns the firmware version for the specified module. - The firmware version is specific to the device and the definition can be found in the device manual. - User can use the firmware version to understand the firmware revision of the module which can be helpful for troubleshooting, support and firmware update.
12	Read_Module_Info_SerialNo (ushort moduleNo)	- Read module info - serial number. This method returns the serial number for the specified module. - The serial number is specific to the device and the definition can be found in the device manual.

		- User can use the serial number to identify and track the module which can be helpful for inventory management, troubleshooting and support.
13	Read_Module_Info_Product_Key (ushort moduleNo)	- Read module info - product key. This method returns the product key for the specified module. - The product key is specific to the device and the definition can be found in the device manual. - User can use the product key to identify the specific product model of the module which can be helpful for troubleshooting and support.
14	Read_Module_Info_Order_Text (ushort moduleNo)	- Read module info - order text. This method returns the order text for the specified module. - The order text is specific to the device and the definition can be found in the device manual. - User can use the order text to get more information about the module which can be helpful for troubleshooting and support.

4.13. Read IO-Link Mailbox

No.	Method Name	Description
1	Read_IO_Link_ISDU_Status()	- Read IO-Link Mailbox - ISDU status. This method returns the ISDU status for the IO-Link mailbox communication. - The ISDU status indicates the current status of the ISDU communication such as whether there is a new ISDU request, whether the last ISDU request is processed, whether there is an error for ISDU communication, and etc. - The specific definition of the ISDU status can be found in the device manual. User can use the ISDU status to monitor and troubleshoot the IO-Link mailbox communication.
2	Read_IO_Link_Command()	- Read IO-Link Mailbox - read/write command. This method returns the read/write command for the IO-Link mailbox communication. - The read/write command indicates the command type for the ISDU communication such as whether it is a read command or a write command. - The specific definition of the read/write command can be found in the device manual. User can use the read/write command to understand the current ISDU communication and process the ISDU data accordingly.
3	Read_IO_Link_MailBox_ModuleNo()	- Read IO-Link Mailbox - Nuthiser. This method returns the Nuthiser value for the IO-Link mailbox communication. - The Nuthiser value is used for ISDU communication to indicate the number of bytes for the ISDU data.

		- The specific definition of the Nuthiser can be found in the device manual. User can use the Nuthiser value to understand how many bytes of ISDU data are involved in the current ISDU communication and process the ISDU data accordingly.
4	Read_IO_Link_MailBox_ChannelNo()	- Read IO-Link Mailbox - channel. This method returns the channel number for the IO-Link mailbox communication. - The channel number is used to identify the specific channel for the ISDU communication in case there are multiple channels available for the IO-Link mailbox communication. - The specific definition of the channel number can be found in the device manual. User can use the channel number to understand which channel is currently used for the ISDU communication and process the ISDU data accordingly.
5	Read_IO_Link_MailBox_ISDU_Index()	- Read IO-Link Mailbox - ISDU index and sub-index. This method returns the ISDU index and sub-index for the IO-Link mailbox communication. - The ISDU index and sub-index are used to identify the specific ISDU data for the communication. The specific definition of the ISDU index and sub-index can be found in the device manual. - User can use the ISDU index and sub-index to understand which ISDU data is currently involved in the ISDU communication and process the ISDU data accordingly.
6	Read_IO_Link_MailBox_ISDU_SubIndex()	- Read IO-Link Mailbox - ISDU index and sub-index. This method returns the ISDU index and sub-index for the IO-Link mailbox communication. - The ISDU index and sub-index are used to identify the specific ISDU data for the communication. The specific definition of the ISDU index and sub-index can be found in the device manual. - User can use the ISDU index and sub-index to understand which ISDU data is currently involved in the ISDU communication and process the ISDU data accordingly.
7	Read_IO_Link_Mailbox_ISDU_Data_ Length()	- Read IO-Link Mailbox - ISDU data length. This method returns the ISDU data length for the IO-Link mailbox communication. - The ISDU data length indicates how many bytes of ISDU data are involved in the current ISDU communication. - The specific definition of the ISDU data length can be found in the device manual. User can use the ISDU data length to understand how many bytes of ISDU data are involved in the current ISDU communication and process the ISDU data accordingly.
8	Read_All_IO_Link_MailBox_Data()	- Read IO-Link Mailbox - ISDU data. This method returns the ISDU data for the IO-Link mailbox communication. - The ISDU data is the actual data involved in the ISDU communication. The specific definition of the ISDU data can be found in the device manual.

		- User can use the ISDU data to understand the content of the ISDU communication and process it accordingly.
9	Read_Block_Parameter_Mailbox()	- Read block parameter - Mailbox. This method returns the parameters related to the mailbox communication in a block. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the mailbox communication at once which can be more efficient than reading each parameter separately.

4.14. Read block data.

No.	Method Name	Description
1	Read_Block_General_Diagnosis()	- Read block diagnosis - general diagnosis. This method returns the parameters related to the general diagnosis in a block. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the general diagnosis at once which can be more efficient than reading each parameter separately.
2	Read_BLock_System_Parameter()	- Read block system parameter. This method returns the parameters related to the general system parameter in a block. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the general system parameter at once which can be more efficient than reading each parameter separately.
3	Read_Block_Module_Diagnosis (ushort moduleNo)	- Read block diagnosis - module diagnosis. This method returns the parameters related to the module diagnosis in a block for the specified module. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the module diagnosis at once which can be more efficient than reading each parameter separately.
4	Read_Block_General_Information()	- Read block general information. This method returns the parameters related to the general information in a block. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the general information at once which can be more efficient than reading each parameter separately.
5	Read_Block_Module_Information (ushort moduleNo)	- Read block module information. This method returns the parameters related to the module information in a block for the specified module. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple

		parameters related to the module information at once which can be more efficient than reading each parameter separately.
6	Read_Block_IO_Link_MailBox()	- Read block IO-Link Mailbox. This method returns the parameters related to the IO-Link Mailbox communication in a block. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the IO-Link Mailbox communication at once which can be more efficient than reading each parameter separately.
7	Read_Block_Condition_Counter()	- Read block condition counter. This method returns the parameters related to the condition counter in a block. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the condition counter at once which can be more efficient than reading each parameter separately.
8	Read_Block_Module_Vaccum (ushort moduleNo)	- Read block module - vaccum. This method returns the parameters related to the vaccum module in a block for the specified module. - The specific parameters included in the block can be found in the device manual. - User can use this method to read multiple parameters related to the vaccum module at once which can be more efficient than reading each parameter separately.
9	Read_Block_Module_Digital (ushort moduleNo)	- Read block module - digital input. This method returns the parameters related to the digital input module in a block for the specified module. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the digital input module at once which can be more efficient than reading each parameter separately.
10	Read_Block_Module_AnalogIn (ushort moduleNo)	- Read block module - analog input. This method returns the parameters related to the analog input module in a block for the specified module. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the analog input module at once which can be more efficient than reading each parameter separately.
11	Read_Block_Channel_IOLink (ushort moduleNo)	- Read block module - IO-Link. This method returns the parameters related to the IO-Link module in a block for the specified module. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the IO-Link module at once which can be more efficient than reading each parameter separately.

12	Read_Block_PQI_IOLink (ushort moduleNo)	- Read block module - IO-Link. This method returns the parameters related to the IO-Link module in a block for the specified module. - The specific parameters included in the block can be found in the device manual. User can use this method to read multiple parameters related to the IO-Link module at once which can be more efficient than reading each parameter separately.

4.15. Write IO Link Mailbox

No.	Method Name	Description
1	Write_IO_Link_Command (ushort commandNo)	- This method writes the read/write command for the IO-Link mailbox communication.
2	Write_IO_Link_MailBox_ModuleNo (ushort value)	-This method writes the Nuthiser value for the IO-Link mailbox communication.
3	Write_IO_Link_MailBox_ChannelNo (ushort value)	- This method writes the channel number for the IO-Link mailbox communication.
4	Write_IO_Link_MailBox_ISDU_Index (ushort value)	- This method writes the ISDU index and sub-index for the IO-Link mailbox communication.
5	Write_IO_Link_MailBox_ISDU_SubIndex (ushort value)	- This method writes the ISDU index and sub-index for the IO-Link mailbox communication.
6	Write_IO_Link_MailBox_ISDU_Length (ushort value)	- This method writes the ISDU data length for the IO-Link mailbox communication.
7	Write_All_IO_Link_MailBox_Data (byte[] data)	- Writes up to 238 bytes (119 registers) into mailbox starting at register 34007. - The method verifies connection, avoids an unnecessary write by comparing current content, and performs the multi-register write inside a lock.
8	Read_Block_Module_Vaccum (ushort moduleNo)	- Read block module - vaccum. This method returns the parameters related to the vaccum module in a block for the specified module. - The specific parameters included in the block can be found in the device manual. - User can use this method to read multiple parameters related to the vaccum module at once which can be more efficient than reading each parameter separately.
9	Write_ISDU	- Write ISDU data for the specified module, channel, index, and sub-index.

	(ushort moduleNo, ushort channelNo, ushort index, ushort subIndex, CmdType cmdType, byte[] data)	- The method verifies connection, prepares mailbox parameters, writes the data to the mailbox, issues the write command, and polls for completion or error status.

4.16. Write Parameter Mailbox

No.	Method Name	Description
1	Write_Parameter_Mailbox_ModuleNo (ushort value)	- Writes a 16-bit value into mailbox register 10000.
2	Write_Parameter_Mailbox_AP_Para_Low_Word(ushort value)	-This method writes the Nuthiser value for the IO-Link mailbox communication.
3	Write_IO_Link_MailBox_ChannelNo (ushort value)	- Writes a 16-bit value into mailbox register 10001. - Skips the write if the device already contains the requested value and verifies connection.
4	Write_Parameter_Mailbox_AP_Para_Low_Word(ushort value)	- Writes a 16-bit value into mailbox register 10002. - Skips the write if the device already contains the requested value and verifies connection.
5	Write_Parameter_Mailbox_AP_Para_Instance(ushort value)	- Writes a 16-bit value into mailbox register 10002. - Skips the write if the device already contains the requested value and verifies connection.
6	Write_Parameter_Mailbox_Execute_Command(ushort value)	- Writes a 16-bit command into mailbox register 10003. - Skips the write if the device already contains the requested value and verifies connection.
7	Write_Parameter_Mailbox_Data_Length (ushort value)	- Writes a 16-bit data length value into mailbox register 10004. - Skips the write if the device already contains the requested value and verifies connection.
8	Write_Parameter_Mailbox_AP_Para_High_Word(ushort value)	- Writes a 16-bit value into mailbox register 10005. - Skips the write if the device already contains the requested value and verifies connection.
9	Write_AP_Parameter (ushort ModuleNo, ushort AP_Para_Low_Word, ushort AP_Para_High_Word, ushort AP_Para_Instance, byte[] data, ushort Data_Length)	- This method writes the specified data into the mailbox and executes the write command.
10	Write_Parameter_Mailbox_Data (byte[] data)	- Writes up to 250 bytes (125 registers) into mailbox starting at register 10010. - If the provided data is shorter than 1024 bytes it will be zero-padded.

		- The method verifies connection, avoids an unnecessary write by comparing current content, and performs the multi-register write inside a lock.